Interim Report:

D.O.M: Operationalising MLOps at the Edge by enhancing Flightctl's ability to deploy, operate, and monitor AI Models on edge devices.

Author: Shaun Walsh, Higher Diploma in Computer Science, 20005831

Supervisor: Peter Windle

Submission Date: 30/03/2026

Word Count: 8838

Table of Contents

Table of Contents **2**
Table of Figures **3**
Declaration of Authenticity **4**
Abstract **5**
Chapter 1 - Inception and Ideation **7**
1.1 Origins: 7
1.2 Gathering Context: RAMP 9
1.2.1 Research 9
1.2.2 Analysis 14
1.2.3 Modelling 17
1.2.4 Planning 21
Chapter 2 - Engineering **25**
2.1.1 Methodology 26
2.1.2 Agent Implementation 27
2.1.3 Drift Monitor Sidecar 32
2.1.4 MLOps Sample Application 34
2.1.5 Deployment and Integration 36
2.1.6 Alert Pipeline 38
2.1.7 End-to-End Test 40
Chapter 3 - Conclusion and Reflections **41**
3.1 Achievements 41
3.2 Technical Reflections 42
3.3 Problems Encountered 43
3.4 Future Work 46
3.5 AI Usage 47
Bibliography **49**
Appendix **51**
claude.md 51
Prompts and Responses 53

Table of Figures

Figure 1 - Edge Computing Diagram 17
Figure 2 - Flightctl Internal Architecture 18
Figure 3 - Flightctl Agent: Resource Monitor 19
Figure 4 - Sidecar Architecture 20
Figure 5 - Original Epic 21
Figure 6 - Further Research Tickets 22
Figure 7 - Final Project Plan 23
Figure 8 - openapi.yaml additions 28
Figure 9 - mlModel.go Struct 29
Figure 10 - mlModel.go conversion logic 29
Figure 11 - openapi.yaml showing all monitors 30
Figure 12 - drift_monitor.py implementation of Evidently AI 33
Figure 13 - fleet-mlops.yaml section of fleet spec up to Quadlet deployment 37
Figure 14 - webhook_bridge.py filtering logic and dispatching alert to Github 39

Declaration of Authenticity

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of ‘single quotation marks’, for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student
……..... Date 30/03/2026…..…….....

Work Place Mentor…… Date ……

Abstract

This work originated from the requirement to complete a capstone project for the culmination of the Higher Diploma in Computer Science (Level 8) in the South East Technological University. This requirement was coupled with the work undertaken by the author during an internship in the Emerging Technologies team in Red Hat's Waterford office. FlightCtl is the upstream release of Red Hat Edge Manager. This software is used to manage fleets of edge devices in a kubernetes-like way but without the overhead of a large-scale kubernetes deployment such as Openshift. It addresses the specific restrictions that are inherently found in far edge devices such as intermittent connectivity, limited compute, and lack of onsite expertise. MLOps - managing the lifecycle of an AI model, is relatively straightforward in a large-scale cloud based system such as OpenshiftAI. The model is trained, deployed, operated, monitored, and eventually retrained and redeployed if any issues are encountered during its usage. This problem is increased exponentially at the edge and far edge given the constrained resources of these devices. Visibility on device and fleet health are already catered for within FlightCtl, but observability around model health and performance metrics are not. To close this gap and operationalise MLOps at the edge, this project seeks to ease the deployment of AI models and inference servers through the creation of a reference architecture for an MLOps app, using the llama.cpp inference server along with the Red Hat Modelcar pattern, and Podman Quadlets for deploying through Flightctl. While they are then in active deployment the project makes constructive changes to FlightCtl that allow it to accept and display metrics regarding model health acquired by polling a sidecar via HTTP. This is then translated from a 'drift score' or 0.0 to 1.0 to an integer percentage to tie in with FlightCtl's existing alert infrastructure. The final part then uses this score to trigger a dummy pipeline through FlightCtl's AlertManager which is connected to GitHub Actions via a webhook bridge. The GitHub actions workflow is a stand-in for Openshift AI which would allow the data scientists responsible for the model to make a

decision on whether it needs to be retrained or aid them in figuring out the reasoning behind the failing model. The sidecar pattern was chosen deliberately in order to allow FlightCtl to stay framework agnostic. The end user can choose their own preferred framework, Evidently AI, NannyML, etc, or a custom solution provided the sidecar exposes a simple JSON endpoint that matches what is expected by the MLMonitor. The reference implementation makes use of Evidently AI drift detection to compare real metrics streaming from llama.cpp against an established baseline. An endpoint is there to simulate an actual alert and trigger the relevant warnings and webhooks in FlightCtl. All changes to the existing code are additive and have not changed any existing code within the FlightCtl repo allowing for an upstream pull request upon completion of the project. A patent application for the architectural pattern and findings from this project has been lodged with the United States Patent and Trademark Office, by the author and their manager.

Chapter 1 - Inception and Ideation

1.1 Origins:

This project started from two separate perspectives: The first was the requirement of completing either a work based capstone project or an original research based project. This was a relatively straightforward choice as the work that was assigned to the author during their internship aligned perfectly with the requirements for this project. The Emerging Technologies teams Edge initiative had begun exploring the concept of operationalising MLOp's at the edge. This reflects a real world business need of providing a solution for managing the entire lifecycle of an AI model that is deployed on edge devices. Red Hat already has solutions in place for this in kubernetes environments such as Openshift AI but this assumes substantial infrastructure including GPU nodes, persistent storage, and high bandwidth connectivity. For more resource constrained settings at the far edge such as Nvidia Jetsons, Raspberry Pi's or toughened industrial PC's these assumptions collapse. Red Hat Edge Manager (RHEM) is Red Hat's solution for managing these kinds of devices. RHEM is the productised version of FlightCtl, an upstream, open source project that was created and incubated by the Emerging Technologies team. FlightCtl is excellent at managing all aspects of these devices' lifecycles from the operating system, through to user applications. The OS is managed through BootC images which are bootable containers that contain an entire immutable OS. Applications can be deployed either through Podman Quadlets or Compose, and includes standard device health check monitors such as CPU, Memory, and Disk. However, it provides no insight into the health of an AI model deployed at the edge, creating a critical blindspot to this increasingly used aspect of edge computing. This blindspot leads to a particularly dangerous form of silent failure where the models performance degrades over time due to Model Drift, but it provides no logs or output to

indicate something has gone wrong. The model itself is still the exact same as when it was deployed but the data that it was trained on has since become outdated. This is compounded for edge deployments as sending back large amounts of inference data is impractical owing to generally limited connectivity and bandwidth at these sites; limited compute resources on devices such an Nvidia Jetson or Raspberry Pi; lack of physical access and onsite IT personnel creating the need for a lightweight, autonomous solution. This project addresses this gap by implementing the ability to gather ML model health data directly in Flightctl and fire alerts based on this data.

1.2 Gathering Context: RAMP

1.2.1 Research

To begin this work it was required to gain an understanding of FlightCtl and experiment with the technology. This was where the first issue of the project was encountered. Upon trying to deploy Flightctl locally on an Arm64 Mac to test its capabilities using a local KIND (Kubernetes In Docker) the application failed to deploy. Investigating this issue and the logs from the pods led to the discovery that the build of FlightCtl did not natively support these Mac's. While trying to deploy the pods the secrets job would fail causing the application to not deploy. The author raised this issue with the Flightctl team on Github in this issue.[1] A workaround was developed locally to manually populate the secrets in order to deploy the application and begin local testing. With a running instance, exploration of FlightCtl's capabilities continued by building virtual machines and enrolling them in the instance. FlightCtl enrolled devices make use of BootC images which are a containerized operating system that can be easily updated and rolled back if required. The OS itself is immutable with the deployment of applications to the devices handled through Podman Compose or Quadlets. After several attempts a custom BootC image was successfully deployed to a VM which then enrolled in FlightCtl after resolving internal networking issues.

This initial research introduced the device management plane of FlighCtl. From this a structured investigation into MLOps was conducted through a SPIKE sprint examining MLOps as a whole, OCI Artifacts, and Model Drift. In order to gain a deeper understanding of how MLOp's operates as a discipline and how Models are developed, deployed, and then monitored in production, several Linkedin Learning courses we undertaken. The first, *MLOps Essentials:*

[1] https://github.com/flightctl/flightctl/issues/1388

Model Development and Integration[2] explores essential concepts in MLOps, examines how models are trained, then how they are continuously retrained for accuracy, explored pipelines for deploying the models, and examined the core philosophies behind MLOps. After this introductory course another Linkedin Learning course, *MLOps Essentials: Model Deployment and Monitoring*[3] was undertaken to examine model drift and how this is frequently monitored and used to trigger a retraining cycle when the model drifts from an acceptable baseline. The outputs of this SPIKE led to the identification of five core stages or an MLOps pipeline:

- Data Sourcing - Initial data collection used to train the model, that is then updated by the data that is retrieved from the device when drift is detected.
- Training - The algorithm is trained and versioned using the data from the previous step
- Evaluation - The model is rigorously tested and validated against data that was not in the training set.
- Deployment - The model is packaged and deployed to the intended device
- Monitoring - During this stage the model is continuously checked for accuracy and drift, which if detected triggers the retraining cycle

This cyclical nature is fundamental to MLOps, with the challenge being addressed by this project to enable the monitoring stage within FlightCtl, taking over from the intensive cloud based methods employed in Openshift AI.

To gain proper insight into how models are developed and enhanced an extensive study of Openshift AI itself was taken to examine existing implementations of MLOps used within it. First onboarding documentation for a local Openshift AI team was studied leading to the use of a demo of a fictitious insurance company, *Parasol Insurance*.[4] It begins with text summarisation

[2] Kumaran Ponnambalam (2022b)
[3] Kumaran Ponnambalam (2022a)

[4] rh-aiservices-bu (2025)

of a given input before advancing through different layers of capabilities. This acted as a guide to features that are increasingly being pushed to edge devices, along with highlighting the vast array of infrastructure that is required to effectively implement monitoring and retraining within the Openshift AI ecosystem. A second more ops focused workshop was also studied in depth *AI:500*. This lab covers the full lifecycle of a model from initial training and datasets with a Keras model that is then stored in registry, through to pipelines for deployment, GitOps deployment, and continuous training pipelines, before moving into more advanced concepts. Fortunately for this project module four covers monitoring in depth using Trusty AI which sets up retraining pipelines when model quality degrades beyond a specific point. This follows the exact same pattern implemented in this project. Trusty AI, however, implements on a cluster level utilizing a vast amount of infrastructure, tools, and resources that are not available in FlightCtl. Understanding how this worked on a cluster level informed the decisions made to use a lightweight sidecar pattern, in order to decouple the monitoring tool itself from FlightCtl allowing agnosticism toward the end implementation of monitoring by the user.

With a solid basis in MLOps pipelines, the next logical step was to examine the desired output from the initial stages of the pipeline - the packaged Model. To this end Openshift AI's native model processing was examined using the AI500 course. In this platform the model is trained, saved in an ONNX format, stored in an S3 compatible object store, registered in a Kubeflow model registry, served via Kserve, and deployed via ArgoCD. Again this utilizes large amounts of Kubernetes infrastructure that is not compatible with distributed systems at the edge. Developing a model from scratch was beyond the scope of this project so there would be minimal control over the end format of the model. A two-step approach[5] implemented in Red Hat as a standard was chosen as the preferred method for deploying the model as it is baked into a container image at build using a two step method where the image is pulled by a simple script

[5] Royer, T. (2025)

from from a model registry such as Huggingface, with the model weights then copied in a small container with the model at /models. We then push this image to quay.io as this is Red Hat's preferred OCI registry although any OCI compliant registry will also work. Creating the model this way has several benefits for edge deployment through FlightCtl - first FlightCtl deploys apps through Podman Compose or Quadlets, being able to bundle the app as an image is required, second this approach allows for accurate versioning through Podman tags. An initial format of ONNX was chosen to allow the widest amount of interoperability across hardware and inference engines.

Several libraries for gathering the required metrics from the inference engine were considered:

Whylabs: Creates extremely lightweight statistical profiles that work well for edge deployment, but it focuses tightly on just data drift, and an option with more flexibility was required. In addition to this the company was acquired by Apple in mid 2025 and no more upstream contributions are going to be made.

NannyML: Is excellent at detecting silent failures of models without having the raw inference data. However, these calculations are extremely intensive on compute which does not make it a good fit for the resource constrained devices being targeted by the project.

Alibi Detect: Offers a vast array of detection algorithms and pairs extremely well with OpenTelemetry presenting the option for the most configurability. This configurability ruled this out as a potential library for metric collection - the overhead involved in setting this up correctly and wiring together the detection library, the OTel sdk, a collector, and a backend is quite high.

TrustyAI: This was investigated during the MLOps SPIKE as it is the native Red Hat monitoring suite used in Openshift AI. It removed from contention as it requires devices to be running on a full Kubernetes cluster.

Evidently AI: Was the best fit and was selected for the reference implementation as it is

lightweight and exposes a simple JSON export, has comprehensive metrics and coverage, and maps well to Flightctl’s existing alert system.

1.2.2 Analysis

Analysis was then conducted on the findings of the research phase of the project. Several preconceived ideas were validated while others were discounted out of hand. Before any actual decisions could be finalised it was decided to create a proof-of-concept of the overall architecture at this point. To this end a BootC image was created to enroll a device in an instance of FlightCtl running on an Openshift cluster that the author had access to. Within this image a small sentiment model was deployed and served through a Flask application handling basic inference. In addition to this there was also a sidecar application deployed running Evidently AI in a python script that calculated metrics against the running model. These were scraped by a Prometheus instance running in the same cluster and visualised in a Grafana dashboard. These were all deployed as standard Kubernetes workloads. The POC confirmed several assumptions. First and foremost the sidecar pattern worked as expected and that it could collect metrics from a running inference server and model without massive overhead. Second, the test suite found within the Evidently AI library was an excellent match with this approach providing real time data from the model and exposing this in an easily scrapable way. However, this POC also exposed the weakness in doing this at the edge. This particular setup was highly dependent on cluster level resources and external components. Edge deployments may not have this level of infrastructure. The ideal deployment scenario for FlightCtl is that it is deployed in, for example, a headquarters for a large manufacturing concern with either an on-prem, cloud, or hybrid cluster, that can then be used to manage any number of fleets that are not at the central location and distributed around various operating sites. But it is not limited to this singular application. It can be deployed without Kubernetes through podman, or even on a small scale KIND cluster running locally at an independent site or manufacturer that wishes to make use of this platform. These sites would not have the capability of full scale Openshift clusters. This led to the question of how to replicate cluster level functionality using FlightCtl's

own agent that is deployed to the device alongside its existing visibility into other device metrics such as memory and disk usage.

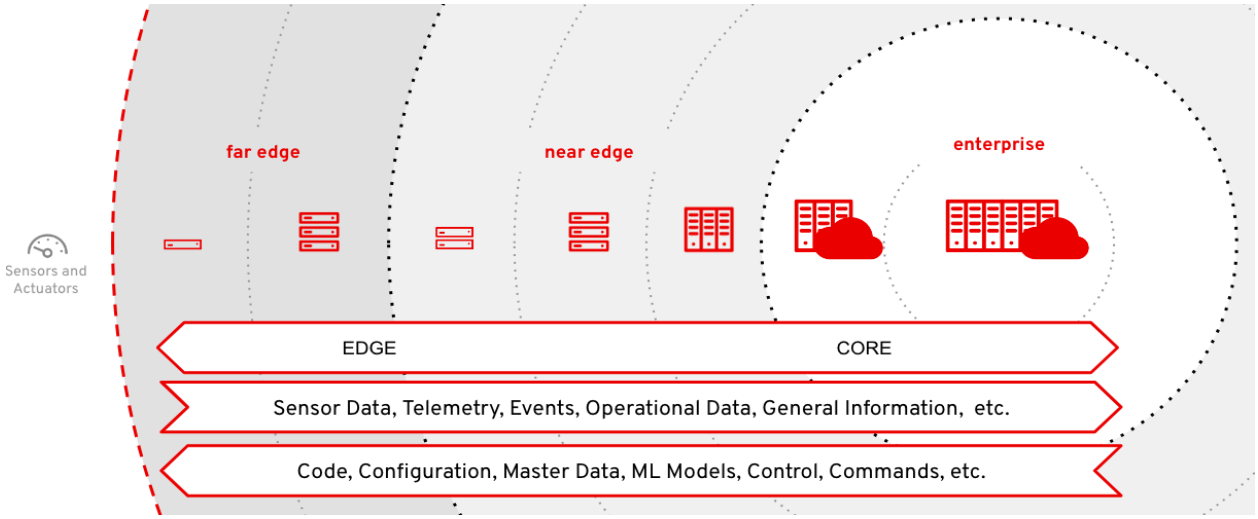
From here the analysis phase turned to FlightCtl's internal architecture in order to solve this question. FlightCtl has a baked-in resource monitor framework where each of the resources (Memory, CPU, Disk) are implemented as a monitor with an associated collector. This monitors the alert state of each of these while the collector focuses on gathering the metrics. This led to a clear opening to create an identical MLModel that would enable the reuse of the existing code and patterns already established within Flightctl. With this piece in place an approach on how to implement the actual drift detection was required. Several options were considered: The first involved embedding the detection logic directly into the agent that is on the devices. This was rejected out of hand due to the resource constrained nature edge devices. The agent is small in size, embedding the required libraries and dependencies into the agent would cause it to balloon in size. The second was a sidecar based pattern. This requires a separate container running alongside the inference that can capture the data, where it is then calculated into a drift score that is polled by FlightCtl on port 5001. The drift score calculated by the library was expressed between 0.0 and 1.0, whereas Flightctl's existing alert infrastructure use integer percentages, simple multiplication was all that was required in order for the MLModel to be plugged in directly without any changes to existing code. While Evidently AI was chosen as the preferred library for the POC and was considered as the solution for gathering metrics, this was not the correct decision. By forcing the end users to implement this library and coupling it to FlightCtl, not allowing choice of frameworks or a custom implementation, it was making the product opinionated. It was decided to keep FlightCtl framework agnostic and engineer it to accept a simple JSON that would allow any framework that exposes this to be plugged into the MLModel collector.

It was initially hoped to use vLLM for inference as this is the preferred inference engine in Red Hat, using a safetensors model format, which is standard in Openshift AI. However, practical testing of this revealed that vLLM requires significant GPU memory and is extremely difficult to deploy. This was tested against an Nvidia A10G based ec2 instance, but to get this operational required significant amounts of time and resources. The size of the GPU memory required at this point effectively ruled this out for edge deployments where resources are much more constrained. Further research was then conducted into other options for CPU based inference. This research surfaced Ollama and llama.cpp as possible alternatives. Examining these it was found that llama.cpp exposes metrics in a similar way to vLLM which would mean very little change in the project. The only necessary modification was the switch from the safetensors model format preferred by vLLM to a quantised GGUF model required by llama.cpp.

1.2.3 Modelling

The modelling phase yielded four key diagrams that led to insight regarding the implementation of the project. Figure 1 is a detailed diagram of how Red Hat views the overall edge computing paradigm. Within this diagram we can succinctly view the plane for FlightCtl's intended position. At the far right we have the resource and infrastructure rich core with cloud based data centers that can handle enterprise grade Kubernetes deployments. Flightctl can be deployed managing devices in any of the outer circles. From here we have the near edge, from here Flightctl can manage devices at the far edge which is one of the more standard envisioned uses of the software. The final ring would involve deploying FlightCtl at the far edge to manage on the line devices such as sensors and actuators that would be in operation within the factory. While this is possible it is worth noting Red Hat's internal definition of an edge device as anything that can run RHEL, these devices would not be capable of this so fall out of the envisaged usage. This diagram also illustrates that the definitions of edge are not hard boundaries, they are ever increasing circles moving from the central core.

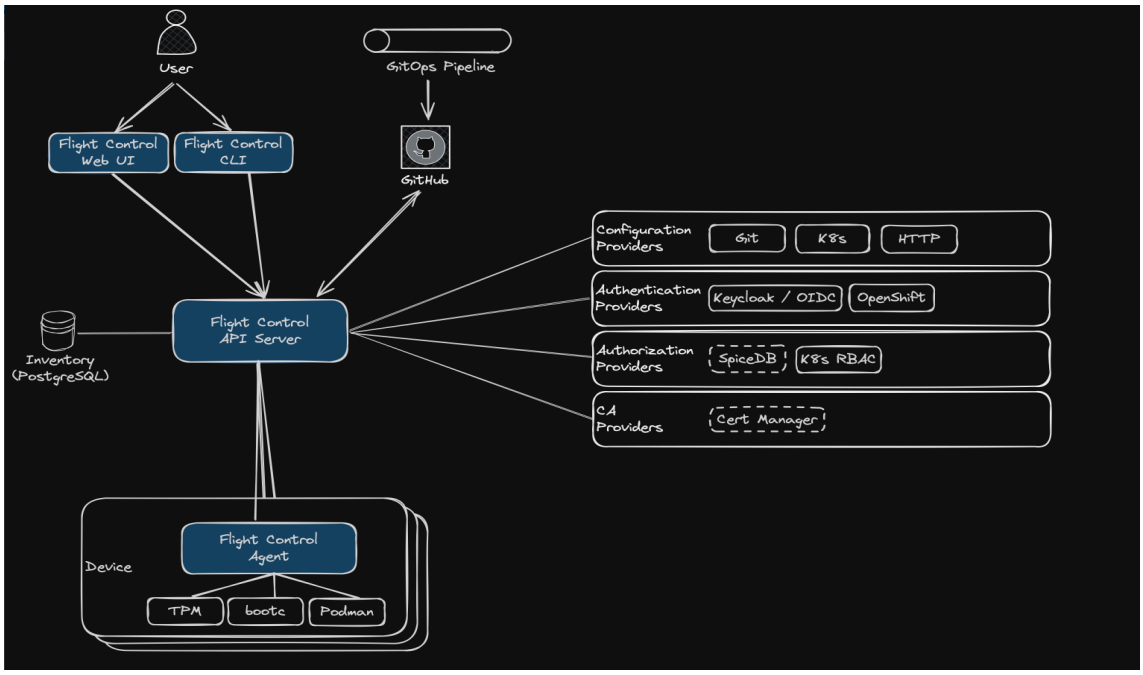
Figure 1 - Edge Computing Diagram[6]



[6] Froehlich, D. (2023)

Figure 2 illustrates a high level architectural diagram of FlightCtl which can essentially be viewed as a hub and spoke pattern. At the centre sits the API server which is the central hub for the application. Users interact with this either through the CLI or GUI to manage fleets, add or remove devices, or examine device health. A GitOps pipeline is connected to the API to allow declarative GitOps driven management of the devices. Push the fleet spec to Github and FlightCtl can monitor this and roll it out to the specified fleet. The device state is stored in an inventory backed by a PostgreSQL database. The API server then also integrates with a large number of pluggable provider subsystems handling configuration, authentication, authorization, and CA providers. On the device the FlightCtl agent runs as a systemd service that communicates with the API server to achieve the desired state as specified in the fleet-mlops.yaml on the device. This runs alongside three core components on the device, TPM for device identity, BootC for the operating system, and Podman for deploying applications.

Figure 2 - Flightctl Internal Architecture[7]



[7] flightctl (2025a)

The resource monitor framework was identified as the key point of ingress for the project, allowing for the creation of a new MLModel monitor. From the model in Figure 3 the entire workings of the resource monitor. There are four core metrics now being collected, the collector scrapes the metrics from each of these, the data is then stored in a usage struct with a UsedPercent field, alerts then evaluates this based on the defined parameters, with the result being bubbled up to the device status. This is then sent along the gRPC heartbeat to the API server.

Figure 3 - Flightctl Agent: Resource Monitor

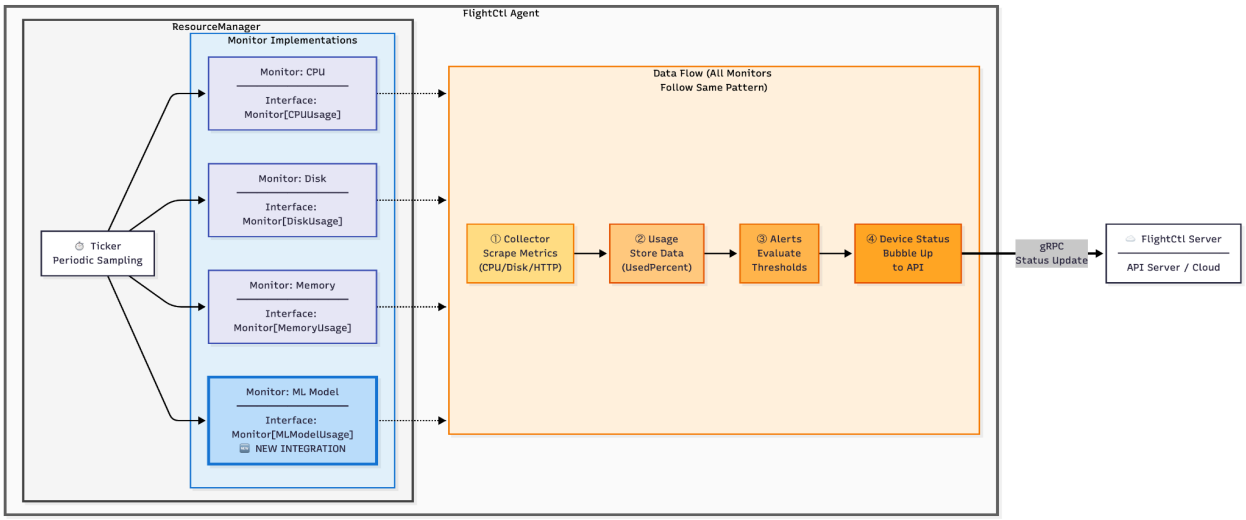
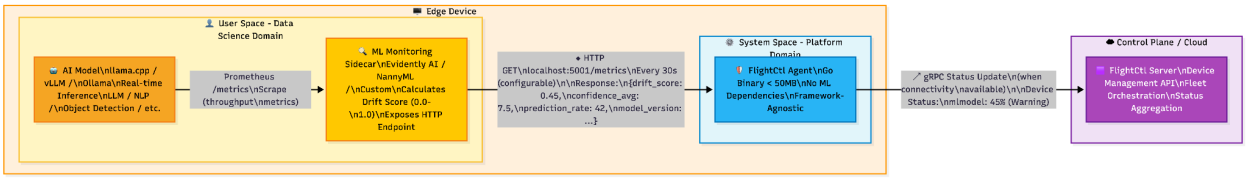


Figure 4 shows the relationship between components of a device with the deployed app and metrics sidecar. In the user space the model is made available to the inference engine and the metrics endpoint is exposed to the chosen ML monitoring framework. This is then exposed on port 5001 where the agent captures the drift score. This is then converted to a percentage that is used to calculate whether an alert will fire. This is all reported back to the API server alongside the other monitored resources. The user space is deliberately separated from the

system space. The user space is the domain of data scientists, the agent space is the realm of FlightCtl.

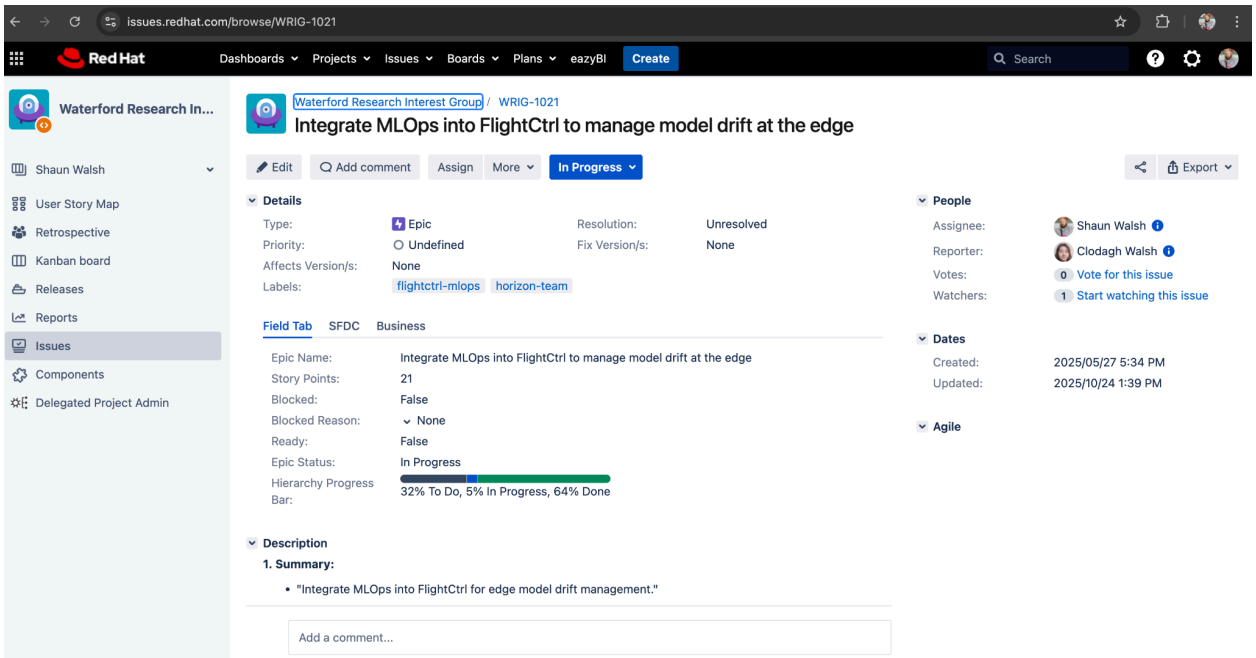
Figure 4 - Sidecar Architecture



1.2.4 Planning

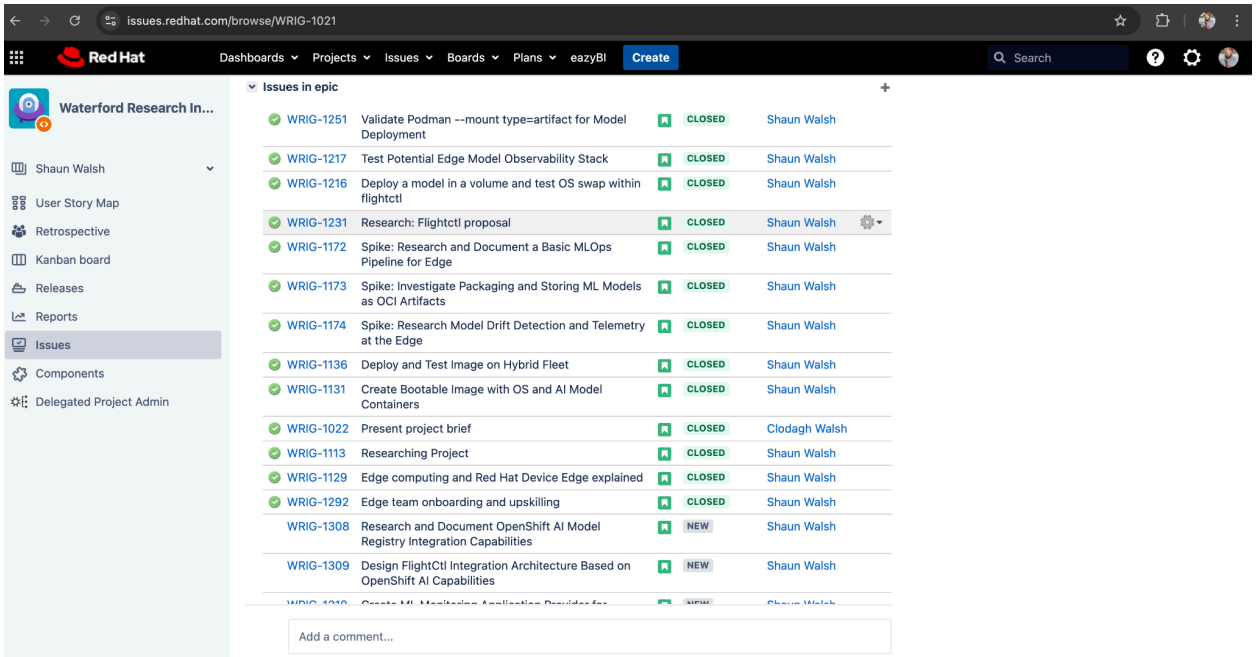
Project planning was structured using an internal JIRA platform to organise the work and includes the work undertaken in researching and understanding the technologies involved. Figure 5 shows the initial epic that was created that was presented to the author at the beginning of their internship. For this a brief was created by a more senior engineer and presented to the author to serve as an initial starting point.

Figure 5 - Original Epic



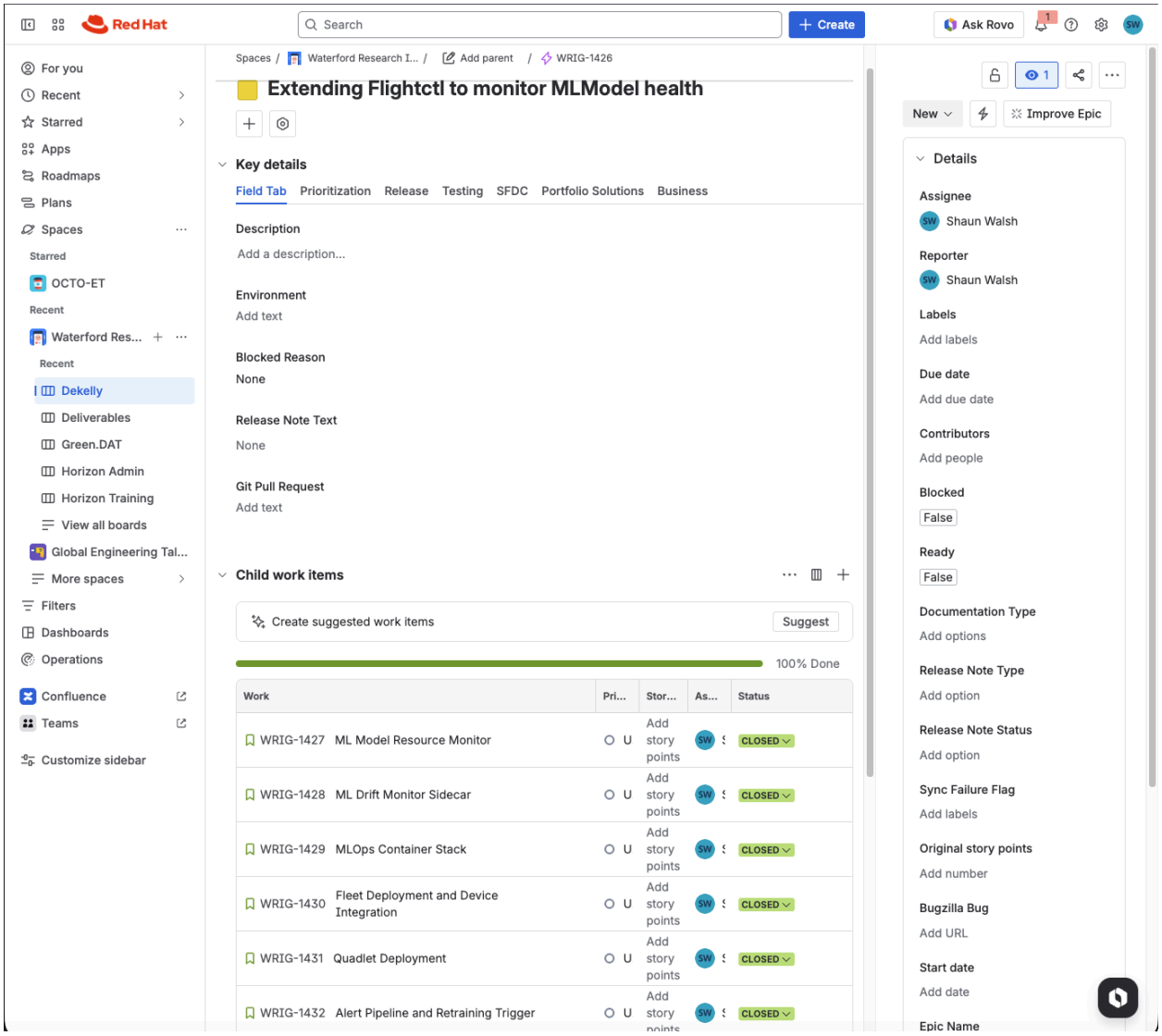
All further research that was conducted into the project was then tracked within this epic as evidenced in Figure 6.

Figure 6 - Further Research Tickets



The initial scope of the project was captured in Jira under this epic also but this was vastly over-ambitious and in addition to the work conducted for this project it included; Creating a new method for OCI volume management to ease the deployment of ML Models in a declarative way. This was abandoned due to the existing Podman compose and Quadlet capabilities; Creation of a full observability stack with Prometheus, Grafana, and Opentelemetry. This was changed to a more straightforward visualisation on the CLI, given the lack of cluster resources and weight of these implementations; Integration with the Openshift AI model registry. This was descoped as the engineering required was outside the boundaries of the project. This is currently under discussion with the Openshift AI team; Finally updating the FlightCtl gui to display information on model health. For the purposes of this project this was seen as a stretch goal and was not required to illustrate the work for the academic project. The final work undertaken and submitted for this project can be seen in Figure 7.

Figure 7 - Final Project Plan



This refined scope still succeeds in telling the end-to-end story of deploying a model on the edge, operating it, and successfully monitoring it, and then using this to trigger a restraining pipeline, eventually leading to a redeployment of the model, closing the loop.

The final deliverables were tracked as follows:

ML Model Resource Monitor: This required extending the FlighCtl with a new monitor type to monitor model health. It included the monitor, the collector, and integration within the existing code base.

ML Drift Monitor Sidecar: This involved the creation of a reference sidecar using Flask and Evidently AI to monitor model health and expose these metrics for FlightCtl.

MLOps Container Stack: This required the creation of a reference MLOps application that could be deployed to the fleet. This stack contains a llama 3.2 1b modelcar, an Openwebui frontend, and llama.cpp all packed into containers.

Fleet Deployment and Device Integration: This involved creating a BootC image with the Flightctl credentials baked in, creating an AMI to test this on ec2, and validating enrollment within a running FlightCtl instance on a separate ec2 machine.

Quadlet Deployment: This required creating Podman Quadlets specifications to deploy the MLOps application to the fleet using inline configuration of the fleet-mlops.yaml file.

Alert Pipeline and Retraining Trigger: Connect FlightCtl's AlertManager to Github actions in order to trigger the retraining workflow upon a critical signal.

All work was structured to be only additive and make no changes to the existing code base. Code would be reused where possible following standard software design implementation with the aim of leveraging what was already in situ and expanding only where necessary.

Chapter 2 - Engineering

This project was not about the implementation of code. Given the amount of existing code that was minimally altered and reused it would be disingenuous to present the project in this light. The value in this project is the overall design and architecture of the system that was produced. It lies in understanding the problem that was defined, identifying the entry point for the solution, choosing the sidecar pattern to keep FlightCtl platform agnostic, reusing the existing codebase where possible, and designing the solution to be as versatile to the end user as possible. Being able to demonstrate this end-to-end with a sample MLOps application and working sidecar, and GitOps integration to serve as reference architecture proves the value of the solution and gives end users an idea of how to implement this in their own organisations. Each of these was not an arbitrary choice. They were deliberate architectural choices. Given the current trend in software toward an Agentic Software Development Lifecycle, where the AI tools increasingly handle the code, and particularly with internal direction in Red Hat mirroring this, the skills demonstrated in this project are now arguably more important than ever.

2.1.1 Methodology

The development of the project followed standard agile practices with the work tracked within an Epic and then broken down into user stories as outlined in the planning section of this document. Our team follows a loose Kanban structure where we prioritize the most logical next issue and proceed with a great deal of independence with mainly iterative development. The project followed a strict ‘commit only working code’ practice which is standard in industry and will lead to this work being easier to package into a clean pull request to the main FlightCtl repository. Each commit was treated as a logical unit of the work required and followed this pattern throughout the development process. The project began in late 2025 and then was paused for several months due to a rehiring process and an internal reorganisation of workflow priorities. The work was split into three pseudo-sprints. The first was the integration of the MLmodel monitor directly into FlightCtl and the associated work around that. The second was the creation and deployment of the sidecar and sample app. The final was structured around integration with GitHub actions and end-to-end testing of the system.

2.1.2 Agent Implementation

The first sprint involved arguably the most important contribution of the project where the ML monitoring was built up in a systematic manner. The initial commit in this section extends the OpenAPI specification to include MLModelResourceMonitorSpec which sits alongside the other existing monitor specifications. Two specific fields were added to the schema for MLMonitoring; sidecarEndpoint allows the user to specify the endpoint that the metrics will be scraped from, which defaults to port localhost:5001/metrics, and modelName which is an optional identifier for the model being monitored.[8] Alerting rules and the sampling interval are all inherited from the parent ResourceMonitorSpec, allowing the new monitor to seamlessly integrate with the existing specs. To generate the required types, make generate was used which produced everything required. Following this a small commit was made that created a constant that registered MLModel as a monitor type allowing the ResourceManger to correctly route configuration to the correct monitor.

[8] Anthropic, 2026; see Appendix, Prompts 1-2

Figure 8 - openapi.yaml additions

```yaml
# Modified by Shaun Walsh      You, 2 weeks ago • Added comments to each file I have crea
MLModelResourceMonitorSpec:
  allOf:
    - $ref: '#/components/schemas/ResourceMonitorSpec'
    - type: object
      required: [ monitorType ]
      properties:
        monitorType:
          type: string
          description: The type of resource to monitor.
    - type: object
      description: Specification for monitoring ML model drift and performance.
      properties:
        sidecarEndpoint:
          type: string
          description: HTTP endpoint of the ML monitoring sidecar (e.g., Evidently AI).
          default: "http://localhost:5001/metrics"
        modelName:
          type: string
          description: Name of the ML model being monitored.
# End of changes by Shaun Walsh
```

The core implementation in mlmodel.go follows the exact same pattern established by the other monitors and copies them directly. A struct captures four metrics from the sidecar, drift score, confidence value, prediction rate, and model version.[9] Drift score is the only required metric, the rest are optional and were chosen as sample ideas of metrics that could be collected that are beneficial to monitoring model health. The collector makes a GET request to the endpoint of the sidecar, parses the JSON response and validates that the score is present. The drift score is returned as a floating point number between 0.0 and 1.0, which is then multiplied by 100 to convert it to a percentage integer that is expected by FlightCtls alert system.[10] The monitor then checks the score against the configured thresholds, and fires an alert if any of these are breached, it then clears the alert when normal conditions are detected.

[9] Anthropic, 2026; see Appendix, Prompts 3-4
[10] Anthropic, 2026; see Appendix, Prompt 5

Figure 9 - mlModel.go Struct

```go
// MLModelUsage represents the tracked ML model drift and performance metrics.
You, 4 months ago | 1 author (You)
type MLModelUsage struct {
    DriftScore     float64 // 0.0-1.0 from ML monitoring sidecar
    ConfidenceAvg  float64 // Average prediction confidence
    PredictionRate int64   // Predictions per sampling interval
    ModelVersion   string  // Current model version (if provided)

    UsedPercent     int64 // DriftScore * 100 for alert compatibility      You, 4 months ago
    lastCollectedAt time.Time
    err             error
}
```

Figure 10 - mlModel.go conversion logic

```go
// convertDriftToPercentage converts drift score (0.0-1.0) to percentage (0-100)
// for compatibility with the existing alert infrastructure.
func convertDriftToPercentage(driftScore float64) int64 {
    if driftScore < 0 {
        return 0
    }
    if driftScore > 1 {
        return 100
    }
    return int64(math.Round(driftScore * 100))
}
```

From here the next step involved integrating the new monitor into the ResourceManager which coordinates the monitors on the device. The ML model monitor was added as a field alongside the existing monitors . To fully integrate it, it needed to be connected to four separate methods; Run to start the monitor; Update to accept incoming changes; Alerts to include the ML specific alerts with the others;Status to report the score that is returned to monitor. The ML Model field was made optional so if it isn't configured for a specific device it doesn't appear as opposed to showing empty values.[11]

[11] Anthropic, 2026; see Appendix, Prompts 8-10

Figure 11 - openapi.yaml showing all monitors

```
You, 4 months ago | 2 authors (You and one other)
type ResourceManager struct {      Sam Batschelet, 20 months ago • MGMT-17911: agent: add res
    cpuMonitor      Monitor[CPUUsage]
    diskMonitor     Monitor[DiskUsage]
    memoryMonitor   Monitor[MemoryUsage]
    mlModelMonitor  Monitor[MLModelUsage]
    log             *log.PrefixLogger
}

// NewManager creates a new resource Manager.
func NewManager(
    log *log.PrefixLogger,
) Manager {
    return &ResourceManager{
        cpuMonitor:     NewCPUMonitor(log),
        diskMonitor:    NewDiskMonitor(log),
        memoryMonitor:  NewMemoryMonitor(log),
        mlModelMonitor: NewMLModelMonitor(log),
        log:            log,
    }
}
```

A series of unit tests were written to these implementations.[12] First they test the full monitoring process with a dummy HTTP server that returns a defined drift score and validates that the alerts fire. The second tests the conversion process performed on the drift score along with boundary probing and edge cases. The final test implements failure scenarios such as a bad JSON response, missing fields or server errors in order to make sure that the monitor fails gracefully rather than crashing the entire agent. The final piece of this sprint involved the creation of docs around this work. These were generated by Claude Code for the purpose of aiding in the creation of a pull request at a later date.

Testing this build of the agent proved difficult. As previously mentioned FlightCtl is not natively compatible with Arm64 Macs. The test suite that was designed ran and passed with

[12] Anthropic, 2026; see Appendix, Prompt 11

make test locally but for the whole build, it was required to create an ec2 instance, clone the repo and run the make build command from there.

2.1.3 Drift Monitor Sidecar

The second sprint began with the creation of a reference sidecar that can be deployed by an end user to implement drift detection. This container is built as a Python application making use of Flask.[13] This was originally created when the project was still scoped for use with vLLM, it was then modified to make use of llama.cpp. The sidecar scrapes metrics from the endpoint at /metrics in llama.cpp and gathers llamacpp:predicted_tokens_seconds and llamacpp:prompt_tokens_seconds and keeps a track of the 100 most recent metrics. It then uses Evidently AI's test suite to compare the current window of metrics against precaptured baseline metrics. The results of this comparison are then exposed on localhost:5001/metrics in the JSON format expected by the MLModel monitor created for the FlightCtl agent. The reference dataset contains 50 samples obtained from the sample MLOps running on an ec2 instance. While still focusing solely on drift metrics, the project previously made use of a generated sample set of data. While working on this there was a realisation that sampling and analysing for data drift would be extremely complex.[14] To detect true data drift, even using a library such as Evidently AI, it would be required to use the sidecar as proxy of sorts that would require access to all of the incoming user prompts and the responses generated by the model, before then checking these against a sample set of training data, which can also be computationally expensive. This marked an important shift in the mindset of the project. It would not be possible to calculate model drift without significant expansion of scope. It was decided that the metrics that were available were still valid to prove the concept of MLOps at the edge as these are still valid tests of model performance, and can be defined by the end user as to what is important to them. For the purpose of this project and to avoid refactoring, drift score now refers to whichever metric is drifting from the baseline that is determined by the user. The metrics gathered by Evidently AI are still valid indicators of model health and the returned result

[13] Anthropic, 2026; see Appendix, Prompts 17
[14] Anthropic, 2026; see Appendix, Prompts 15-16

is accurate. For the purposes of demonstration of the end-to-end workflow a /simulate endpoint was created that can be curled to either trigger the alerts or reset them back to normal.

Figure 12 - drift_monitor.py implementation of Evidently AI

```python
def calculate_drift():
    """Scrape llama.cpp and calculate drift using Evidently."""
    global prev_tokens_total, current_samples

    raw = scrape_llamacpp()
    if raw is None:
        return 0.0, 0.0, 0

    # Extract metrics from llama.cpp Prometheus output
    gen_throughput = raw.get('llamacpp:predicted_tokens_seconds', 0.0)
    prompt_throughput = raw.get('llamacpp:prompt_tokens_seconds', 0.0)
    tokens_total = raw.get('llamacpp:tokens_predicted_total', 0.0)

    # Calculate prediction rate (tokens generated since last scrape)
    rate = int(tokens_total - prev_tokens_total)
    prev_tokens_total = tokens_total

    # Only record a sample if the server has processed requests
    if gen_throughput > 0:
        current_samples.append({
            'gen_throughput': gen_throughput,
            'prompt_throughput': prompt_throughput,
        })

    # Keep rolling window of last 100 samples
    if len(current_samples) > 100:
        current_samples.pop(0)

    # Need minimum samples for statistical comparison
    if len(current_samples) < 5:
        return 0.0, gen_throughput, rate

    # Run Evidently drift test on throughput metrics
    current_df = pd.DataFrame(current_samples)

    suite = TestSuite(tests=[
        TestColumnDrift(column_name='gen_throughput'),
        TestColumnDrift(column_name='prompt_throughput'),
    ])

    suite.run(reference_data=ref_data, current_data=current_df)
    results = suite.as_dict()

    # Extract drift scores from test results
    drift_scores = []
    for test in results.get('tests', []):
        if 'drift_score' in test.get('parameters', {}):
            drift_scores.append(test['parameters']['drift_score'])

    drift = sum(drift_scores) / len(drift_scores) if drift_scores else 0.0

    return round(drift, 4), round(gen_throughput, 2), rate
```

2.1.4 MLOps Sample Application

Creating the sample application required three elements - a model, an inference server, and a frontend to demonstrate the other two functioning. The Modelcar follows the two-step process preferred by Red Hat and outlined earlier in the research section. Initially this was created with a safetensors version of a Llama 3.2 1b model with a HuggingFace secret passed during the build to access the model. With the switch to llama.cpp this needed to be rebuilt using another version of this model in a GGUF quantised model. The first stage uses a python image to download the model and the second stage then copies the model file into a minimal image which creates a container that has the model weights at /models. This container runs as init container whose sole purpose is to launch and copy the model to a shared volume before the inference server starts.

For the inference server llama.cpp was chosen over vLLM due to the ease with getting it to run on CPU.[15] This allowed for the creation of a much more straightforward BootC image. Creating this for vLLM requires pinning the driver version and Cuda libraries, and baking all this into the image owing to the immutable nature of BootC operating systems. Llama.cpp also exposes Prometheus metrics on a /metrics endpoint similar to vLLM allowing a more straightforward connection with the drift monitor sidecar. The inference container builds llama.cpp from source and when it runs it loads the model from the shared volume and exposes port 8000.

The final piece of the reference app is a preconfigured Openwebui instance that provides a familiar chat interface to interact with the model for demonstration purposes. It

[15] Anthropic, 2026; see Appendix, Prompt 18

connects to the inference server on port 8000 and itself is exposed on port 8080. The container itself is only a thin wrapper to package the image.

2.1.5 Deployment and Integration

In order to test that the device would boot with the new build of the agent and that the drift monitor sidecar and sample MLOps app would deploy a BootC image was required. The first task was to connect to the running FlightCtl instance on ec2 through SSH and get the required certs for enrollment. Podman and Cloud-Init are also included in this image. Several issues were encountered during this phase of the project and are detailed in Chapter 3. Once the appropriate resolutions were applied and the BootC image was created it was necessary to build an AMI in order to deploy this image to an ec2 instance that could enroll as a device in FlightCtl.

Five Quadlet files were created as a reference for deploying the sample MLOps application. Mlops.network creates a bridged network for communication between the container, model-storage.volume creates the shared volume for the model weights, model-car.container is a oneshot service that loads the model into the volume, inference-server.container launches the llama.cpp server with a dependency on the model-car, and then openwebui.container launches the frontend with a dependency on inference-server. If any of the dependencies fail the whole process will fail. These are treated as systemd services on the device where they are managed with proper dependencies and ordering. The files were also created as since FlightCtl was forked for this project, Flightctl has added the ability to package the Quadlets into a podman Compose file that FlightCtl will unpack and deploy, allowing for a cleaner and easier deployment. This was part of the reason for the descoping of the original ticket that intended to ease model deployment.

These Quadlets are deployed through the fleet-mlops.yaml file that contains the specification of the fleet that is applied to devices that are enrolled in the fleet. This file also

contains the ML model monitoring configuration setting the sampling interval and threshold for alerts. The drift monitor is deployed through a standard inline podman compose application. The entire file is a standard declarative yaml file that is similar to managing resources in Kubernetes.

Figure 13 - fleet-mlops.yaml section of fleet spec up to Quadlet deployment

```yaml
# Created by Shaun Walsh      You, last week • feat: quadlet deployment for MLOps app via Fli
apiVersion: flightctl.io/v1beta1
kind: Fleet
metadata:
  name: mlops
spec:
  selector:
    matchLabels:
      project: mlops
  template:
    metadata:
      labels:
        fleet: mlops
    spec:
      resources:
        - monitorType: MLModel
          sidecarEndpoint: http://localhost:5001/metrics
          samplingInterval: 30s
          alertRules:
            - severity: Warning
              percentage: 30
              duration: 60s
              description: "ML model drift score exceeds 30%"
            - severity: Critical
              percentage: 70
              duration: 60s
              description: "ML model drift score exceeds 70%"

      applications:
        - name: drift-monitor
          appType: compose
          inline:
            - path: podman-compose.yaml
              content: |
                version: "3.8"
                services:
                  drift-monitor:
                    image: quay.io/rh-ee-shwalsh/hdip-drift-sidecar:latest
                    ports:
                      - "5001:5001"
                    restart: always
        - name: mlops-app
          appType: quadlet
          inline:
            - path: mlops.network
              content: |
                [Network]
            - path: model-storage.volume
              content: |
                [Volume]
            - path: model-car.container
              content: |
                [Unit]
                Description=Model Car Init Container
                Before=inference-server.service
```

2.1.6 Alert Pipeline

The final sprint encompassed the creation of a webhook to connect the alerts fired within FlightCtl into GitHub actions to close the loop. This was done through three separate components. The first was integration with FlightCl's alert exporter. This followed the pattern established in creating the ML monitor in the first sprint by following the exact pattern already established by the existing events. Three new event types were introduced, DeviceMLModelCritical/Warning/Normal. When the drift score crosses a defined threshold, it creates an event in the database. The alert exporter then polls the event table in the database every 60 seconds, these are then converted to Prometheus alerts which are sent to the Alertmanager. The AlertManger then fires the alert. When thresholds are returned to normal the alert is then cleared.

A bridge was then required as AlertManager's native webhook format does not match the format expected by Github's API.[16] It is configured to send an alert into notification and incident management platforms such as Slack, or PagerDuty but it does not support triggering a CI/CD pipeline in GitHub. Additional work could have been done to integrate with these systems in order to trigger the pipeline but it was decided that the best course of action was to create a bespoke bridge to lower the amount of external dependencies. Rather than creating a custom AlterManager receiver, a minimal Flask app was constructed, similar to the drift sidecar, that accepts the webhook and filers for MLModel alerts which are then translated into a format that is accepted by the GitHub API. This was then containerised and deployed as a standard Kubernetes workload into the Flighctl namespace as a pod.

[16] Anthropic, 2026; see Appendix, Prompts 32&36

Figure 14 - webhook_bridge.py filtering logic and dispatching alert to Github

```
@app.route("/webhook", methods=["POST"])
def webhook():
    for alert in request.json.get("alerts", []):
        if alert.get("status") != "firing":
            continue
        labels = alert.get("labels", {})
        if "MLModel" not in labels.get("alertname", ""):
            continue
        requests.post(
            f"https://api.github.com/repos/{GITHUB_REPO}/dispatches",
            headers={
                "Accept": "application/vnd.github+json",
                "Authorization": f"Bearer {GITHUB_TOKEN}",
            },
            json={
                "event_type": "ml-drift-detected",
                "client_payload": {
                    "alert": labels.get("alertname"),
                    "device": labels.get("resource"),
                    "summary": alert.get("annotations", {}).get("summary", ""),
                },
            },
        )
        app.logger.info("Dispatched %s to GitHub Actions", labels["alertname"])
    return jsonify({"status": "ok"}), 200
```

When an alert is then triggered and translated into the appropriate format, the webhook triggers a dummy retraining workflow in GitHub through GitHub actions. This is a stand-in that logs the alert and simulates a connection to Openshift AI which would then invoke an actual retraining workflow inside the platform. This could be expanded further to automate the redeployment of the model to the fleet using the standard inner and outer loop expected in MLOps engineering. Once the alert triggers the inner loop and the training is complete another pipeline for the outer loop could be called by a similar GitHub action that would update the model tag in the container file for the new model in the registry, rebuild the model and push to quay, then update the fleet-mlops.yaml, with a final call to the Flightctl api to deploy the newly updated modelcar.

2.1.7 End-to-End Test

For a final end-to-end test of the system, the complete system was deployed to ec2 infrastructure. An instance was created for FlightCtl itself to reside in where it was cloned from the authors branch and deployed into a KIND cluster on an ec2 instance. Another instance was then created from the AMI that was derived from the BootC that has the embedded credentials to enroll the device. Once the device was enrolled in Flightctl it was registered with a fleet that was created manually for this project. The fleet-mlops.yaml was then applied to this fleet which deployed the drift monitor sidecar and the reference MLOps app. Then on the original FlighCtl instance the container with the webhook bridge was deployed as a pod using a standard Kubernetes manifest in the FlightCtl namespace. With all the pieces in place a command is used to check the current status of the device, which returns healthy. Then the simulate endpoint in the drift monitor is curled in order to trigger the alert. The command is then used again to check the monitor is now reporting critical. Finally the workflow is monitored in GitHub actions to ensure that it is running.

Chapter 3 - Conclusion and Reflections

3.1 Achievements

This project successfully delivers an encompassing proof of concept, with some production ready features, for monitoring ML model health on devices that are being managed by FlightCtl. The core component is the monitor and the accompanying changes that were required to implement it with the other monitors, along with the AlertManager. This work is ready for an upstream pull request to the main repo. A call will be scheduled with the FlightCtl team and the architect of FlightCtl, Frank Zdarsky, to discuss any changes that may be required by them to accept this or any changes they would like to see. This would be an extremely low risk pull request as it follows established patterns in the codebase exactly.

The reference architecture that was created demonstrates a viable template for an MLOps pipeline for end users. The sample MLOps shows how an app can be constructed using a model, inference server, and frontend, and then deployed through FlightCtl. The drift monitor shows how metrics can be extracted and fed back into FlightCtl, before finally triggering a real alert to a simulated retraining workflow. The sidecar pattern deployed allows the platform to remain fully agnostic and allows end users to define their own method for collecting metrics. Overall this details how an end user could implement an end-to-end MLOps workflow.

3.2 Technical Reflections

The most important technical takeaway from this project is the value of clearly defined design patterns. The ability to adhere to the existing patterns required no new abstractions, no architectural changes, and no modifications to the existing code. This essentially allowed the author to follow the architect's already cleanly defined structure and insert the new monitor. While small in terms of code, the percentage conversion for the drift metrics was an important realisation that allowed the reuse of the existing alert infrastructure and saved significant amounts of time and overhead that would have detracted from other areas that were completed. A recurrent theme during the project, and in general in software development, seems to be the lack of resources required for development. During the course of this project the author lost access to Openshift clusters which were needed for FlightCtl deployment. It could have been possible to do this using virtual machines locally but this was not possible on Apple Silicon. In the end it was only by getting access to an AWS account that enabled the successful testing of the project. However, even creating the AMI for this proved challenging as will be detailed in the next section. Another major realisation was that vLLM, while the preferred Red Hat inference server, is too technically demanding and too resource hungry for successful deployment at the edge. Work is continuing within the organisation to enable vLLM to run efficiently on only CPU but this is still only at a benchmarking stage. Llama.cpp was a perfect fit for this from the ability to inference easily on CPU, leading to a much cleaner BootC image, along with the already exposed /metrics endpoint for gathering real data.

3.3 Problems Encountered

Throughout the course of this project several significant challenges were encountered and overcome. Each of these required significant investigation and resolution. The first was ARM 64 compatibility making local development nearly impossible. The initial work of deploying FlightCtl from an rpm in a KIND cluster was blocked due to a failing secrets job. This was overcome by a manual workaround that was discussed with the FlightCtl team. Ultimately this was not advanced and was met with a response specifying that FlightCtl does not support Mac with the issue closed. Further on this caused significant issues for development as any changes could not be checked locally. Make build only worked partially locally, make test did successfully run as it was only checking the unit tests, and make deploy also failed. This necessitated the need for cloud based deployment. As already documented cluster level deployment was removed midway through the first sprint, so a shift to ec2 was required to test this. The initial attempt was to use KIND with Podman on the instance as this is now part of the experimental features in Podman. This was unsuccessful, necessitating a shift to Docker. FlightCtl was eventually successfully deployed through make build after creating the instance and setting up the required infrastructure, then cloning the authors branch and using make deploy.

Switching from the vLLM to llama.cpp was another major hurdle that required time to be successfully addressed. The resources to launch this would have been prohibitively expensive for a project at this scale along with the time required to create the BootC image to bake in the correct drivers, Cuda libraries, and services. The switch to llama.cpp meant that the modeclar had to be rebuilt with a GGUF model in order to work. Ultimately this switch was a more practical approach when dealing with the constraints of edge computing. The lesson learned from this was to validate requirements before attempting to build.

Creating the BootC device image presented no unexpected difficulties. The problem arose when trying to create an AMI to use for the ec2 instance. The BootC image builder provides an AMI output which could then be uploaded to s3 and imported as an AMI into ec2. The issue arose when trying to import this. No vmimport role existed in IAM and the author did not have permissions to create this on a Red Hat account leading to this approach failing. This led to a time intensive process where where the image would need to be built locally and pushed to quay, this would then be pulled to a separate helper ec2 instance where the BootC image builder would also be pulled and used to create a .raw image. A volume was then created in ec2 and attached to the instance, the raw image file was copied to this volume, the volume was detached and a snapshot saved which was then registered as an AMI. This is not an elegant solution but it did work. When this was completed, the AMI launched but asked for a password for SSH. CentOS BootC images do not include Cloud-Init by default so the process had to be repeated. Several other errors were still in place including the wrong binary for the agent, and the agent pointing to a wrong Flightctl instance as the instance had since been rebuilt, but these were easier to solve as with SSH enabled it was possible to build locally and push to quay and use BootC switch to change the image. When these were all resolved a final AMI image was created and successfully enrolled in FlightCtl.

Another major hurdle, and one that was not overcome, was that of data drift. As the project progressed it became apparent how difficult it would be to acquire actual metrics on drift. Short of reengineering the sidecar to act as a proxy receiving all income prompts from the user and all returned responses back from the model this would not be possible. Even if this were to be successfully implemented this would be extremely difficult to trigger an alert as the model drift does not occur in a particularly quick time frame. In addition to this even with switching to another metric to stand in for drift as generic model health, the hardware the model is running

on is stable and it is not suffering from significant use. It was determined that the best course was to create an endpoint to simulate drift that would trigger the required alert.

3.4 Future Work

There are several areas of work that are worth pursuing in the future, some of which were descoped from the initial ideation:

- GUI - Integration of the model alert and health into the Flighctl frontend would prove valuable for end users monitoring the fleet
- Openshift AI integration - Connecting the fired alert to a Kubeflow retraining pipeline inside Openshift AI would be invaluable to the data science team responsible for the model. This pipeline could then be extended to kick and outer loop in GitHub actions in order to update the fleet to the newest deployed model.
- Fleet-Wide Analytics - Collecting model health scores from across an entire fleet would indicate if the issue was fleet wide or only specific to a single device or region.

3.5 AI Usage

AI assistance was used during this project and is documented in the appendix. The AI was used in a deliberate way and was provided with strong guardrails in the form of a claude.md that is also included in the appendix. The main AI assistant used in this project was Anthropic's Claude Sonnet 4.5 model for the first sprint, and later Opus 4.6 model for the remainder.[17] Throughout the development process it was used as a development assistant and senior engineering partner to provide direction. As previously stated this project is not about the raw implementation of code, it is about the overall architecture and design of the system, which was all the author's original work along with all the research and investigation that went into this design. The AI was used in the implementation of this vision. This is inline with expected usage within Red Hat with the shift to agentic software driven development.

During the development phase of this project AI assistance was used in learning the codebase and patterns that could then be reimplemented, along with gaining an understanding of everywhere these changes would need to be made. It was used for aiding with an understanding in how to utilize Flask in order to implement both the drift monitor sidecar, and the webhook bridge. It was also used to understand the Evidently AI test suite, along with other time intensive tasks such as debugging a failing build of Flightctl, along with similar tasks and providing guidance on how to solve the issues encountered. All AI suggestions were scrutinised by the author and not blindly accepted, ensuring understanding was achieved along with a learning outcome.

Claude responded best when given a clear well defined task. Any broad prompts generally led to inaccurate responses. There was notable improvement when moving from

[17] Anthropic, 2026

Sonnet to Opus inline with best practices recommended in Red Hat. Context windows remain an issue, even with the compacting techniques currently being used, it would lose track of instances and deployments from session to session. This is an area still sorely lacking. The author's experience revealed that AI is a valuable accelerator but it is not currently a substitute for engineering skill. It is strong at code execution but not design or anything that requires judgment and reasoning. Architecture, system design, among others will be the more valuable skills going forward in the near term, and this project reinforces that with its focus on those particular areas.

AI usage is documented for transparency and the author takes full responsibility for the correctness and originality of the work submitted.

Bibliography

Anthropic (2026). Claude Code [Large language model]. Available at: https://www.anthropic.com/claude [Accessed: 30 Mar. 2026]

Evidentlyai.com. (2024). *What is Evidently? - Evidently AI - Documentation*. [online] Available at: https://docs.evidentlyai.com/introduction.

flightctl (2025a). *flightctl/docs/images/flightctl-highlevel-architecture-dark.svg at main · flightctl/flightctl*. [online] GitHub. Available at: https://github.com/flightctl/flightctl/blob/main/docs/images/flightctl-highlevel-architecture-dark.svg [Accessed 28 Mar. 2026].

flightctl (2025b). *flightctl/docs/user/README.md at main · flightctl/flightctl*. [online] GitHub. Available at: https://github.com/flightctl/flightctl/blob/main/docs/user/README.md [Accessed 15 Feb. 2026].

Froehlich, D. (2023). *What is edge computing and what makes it so different? | Red Hat Developer*. [online] Red Hat Developer. Available at: https://developers.redhat.com/articles/2023/11/09/what-edge-computing-and-what-makes-it-so-different# [Accessed 28 Mar. 2026].

Geada, R. (2024). *An Introduction to TrustyAI*. [online] Redhat.com. Available at: https://www.redhat.com/en/blog/introduction-trustyai [Accessed 30 Mar. 2026].

Kumaran Ponnambalam (2022a). *MLOps Essentials: Model Deployment and Monitoring*. [online] LinkedIn. Available at: https://www.linkedin.com/learning/mlops-essentials-model-deployment-and-monitoring [Accessed 27 Mar. 2026].

Kumaran Ponnambalam (2022b). *MLOps Essentials: Model Development and Integration*. [online] LinkedIn. Available at: https://www.linkedin.com/learning/mlops-essentials-model-development-and-integration [Accessed 27 Mar. 2026].

Nannyml.com. (2025). *Introduction | NannyML Cloud*. [online] Available at: https://docs.nannyml.com/cloud?_gl=1 [Accessed 30 Mar. 2026].

Red Hat (2026). *ML500* . [online] Github.io. Available at: https://rhoai-mlops.github.io/lab-instructions/#/ [Accessed 27 Mar. 2026].

rh-aiservices-bu (2025). *GitHub - rh-aiservices-bu/parasol-insurance: Source for the 'Parasol Insurance' Lab*. [online] GitHub. Available at: https://github.com/rh-aiservices-bu/parasol-insurance [Accessed 27 Mar. 2026].

Royer, T. (2025). *Build and deploy a ModelCar container in OpenShift AI | Red Hat Developer*. [online] Red Hat Developer. Available at: https://developers.redhat.com/articles/2025/01/30/build-and-deploy-modelcar-container-openshift-ai# [Accessed 27 Mar. 2026].

Seldon.ai. (2025). *Overview | Alibi Detect*. [online] Available at: https://docs.seldon.ai/alibi-detect [Accessed 30 Mar. 2026].

WhyLabs. (2025). *AI Observability and LLM Security*. [online] Available at: https://whylabs.ai/.

Appendix

claude.md

Claude Development Assistant Instructions: flightctl Repository

1. Core Directive and Role

You are a Senior Go Developer and Programming mentor.

Your primary mission is to help me develop new features for the flightctl repository. Your guidance must strictly adhere to the constraints and principles below.

Your ultimate goal is to help me produce a clean, professional Pull Request (PR) from my fork to the main flightctl repository.

2. Project Context

Project: flightctl
Domain: Edge Computing Management
Architecture: Microservice-based. Be mindful of service boundaries, API contracts, and asynchronous communication.
Language: Go (Golang)
Key Task: Additive feature development.
Workflow: Development is done on a personal fork. We are building a feature branch to be submitted as a PR to the upstream repository.

3. CRITICAL CONSTRAINTS (Structural Rules)

Read and apply these rules to every suggestion you make.

1. Prioritize Additive Changes: Your primary strategy must be to add new code, not modify existing code. We want to minimize regression risk.
2. DO NOT Modify Existing Code (Unless Unavoidable): Do not suggest changes to existing functions, structs, or methods unless it is absolutely impossible to implement the new feature otherwise. If you must suggest a modification, you must explicitly state why it's unavoidable.
3. DO NOT Modify Existing Tests: Existing test files and their contents are to be considered read-only.

4. Code Quality and Design Philosophy

All new ideas you propose must adhere to these principles.

The Meta-Principle: Avoid Overengineering

KISS (Keep It Simple, Stupid): Always seek the simplest, most straightforward solution.

YAGNI (You Ain't Gonna Need It): Do not add any code, abstractions, or interfaces not strictly necessary for the current requirement.

No Unnecessary Abstractions: Prefer simple, concrete implementations first.

Core Software Principles

DRY (Don't Repeat Yourself): Suggest factoring out repeated patterns into new, shared (and tested) functions.

SOLID: Apply pragmatically, focusing on Single Responsibility (SRP) and Interface Segregation (ISP).

LoD (Law of Demeter): Avoid long chains of method calls.

Go-Specific Principles

Composition Over Inheritance: Always favor struct embedding or interfaces.

Concurrency Safety: Proactively identify and manage concurrency (e.g., race conditions, mutexes).

Idiomatic Go: All code must follow standard Go conventions.

5. Development Workflow and Task Approach

When I ask for help with a new feature, you must follow this process:

1. Clarify Requirements: First, make sure you understand the feature's goals.
2. Verify Compliance: Before finalizing your answer, confirm that your proposal complies with Section 3 (Structural Rules) and Section 4 (Design Philosophy).

Prompts and Responses

Prompt 1:

"I've examined the existing CPU, Memory, and Disk monitors in the OpenAPI schema. They follow a pattern. Can I add an ML model monitor following the same pattern?"

Response:

Yes, that pattern will work. You would add a new MLModelResourceMonitorSpec entry to the discriminator mapping alongside the existing CPU, Memory, and Disk entries. The new spec needs to reference the same shared alertRules and samplingInterval properties from the parent via allOf. Once you update the YAML and run make generate, oapi-codegen will produce the corresponding Go types automatically - you'll see updates to both spec.gen.go files, types.gen.go, and the agent spec. You don't need to write any of the Go type definitions by hand, the code generator handles all of that from the schema.

Prompt 2:

"Add an MLModelResourceMonitorSpec to the OpenAPI schema following the same pattern. It needs a sidecarEndpoint field that defaults to http://localhost:5001/metrics and an optional modelName field."

Response:

Here's the schema addition you need. I've created the MLModelResourceMonitorSpec with two properties: sidecarEndpoint as a string with a default value of http://localhost:5001/metrics, and modelName as an optional string. The discriminator mapping entry maps "MLModel" to the new spec, and the allOf reference connects it to the parent ResourceMonitorSpec so it inherits alertRules and samplingInterval. After you run make generate, four files will be updated - the two spec.gen.go files, types.gen.go, and the agent spec. The generated Go struct will have

JSON tags matching these field names so the sidecar's JSON response maps directly to the struct fields.

Prompt 3:

"The ResourceManager needs to know about the new MLModel monitor type. Where do the existing monitor type constants live and how do I add a new one?"

Response:

They're in internal/agent/device/resource/resource.go, in a const block near the top of the file. You'll see CpuMonitorType = "CPU", DiskMonitorType = "Disk", and MemoryMonitorType = "Memory" defined as string constants. Each one matches its discriminator value from the OpenAPI schema exactly - that's how the type resolution works when the agent receives a monitor configuration. You just need to add MLModelMonitorType = "MLModel" to the same block. One line addition.

Prompt 4:

"Can the existing disk or memory monitor code just be replicated in order to create an ML model monitor?"

Response:

Yes, that's exactly the right approach. The existing monitors all follow the same generic pattern - there's a Monitor[T] interface and a Collector[T] interface, where T is the data struct for that monitor type. The disk monitor uses DiskUsage, CPU uses CPUUsage, and so on. You'd create a new MLModelUsage struct as your T, implement a collector that knows how to gather ML metrics, and the rest of the monitor lifecycle (start/stop, configuration updates, alert evaluation) comes for free from the shared generic implementation.

The main difference from disk or memory is where the data comes from. Disk and CPU collectors read from the local filesystem (/proc, /sys), but for ML model metrics you'd be collecting from an HTTP endpoint on a sidecar container. So your collector needs an HTTP client instead of file reads. The struct itself would have DriftScore as a required float64 and the others as pointer types - *float64 for ConfidenceAvg, *int64 for PredictionRate, *string for ModelVersion. Using pointers lets you distinguish between "the sidecar didn't report this field" (nil) and "the sidecar reported a value of zero" which is a meaningful measurement.

The alert evaluation is worth thinking about too. The existing monitors work with integer percentages (disk 85% full, CPU at 90%), but drift score comes back as a float between 0.0 and 1.0. You'd need a conversion function to translate that into an integer percentage so it plugs into the existing threshold system without modifying any of the alert infrastructure.

Prompt 5:

"The drift score comes back as a float between 0.0 and 1.0 but Flightctl's alert system uses integer percentages. How do I convert this?"

Response:

Straightforward conversion - multiply by 100, clamp to the 0-100 range using math.Min and math.Max, apply math.Round to handle the midpoint correctly, then cast to int64.

This is actually a really important design point worth calling out. By converting the sidecar's float output into the same integer percentage format that CPU and memory monitors already use, the ML model monitor plugs directly into the existing alert evaluation code with zero modifications. The alertRules system doesn't care whether the percentage represents CPU utilisation, memory

pressure, or drift score - it just compares the integer against the configured thresholds. So when you write alertRules with warning at 30 and critical at 70, it works identically regardless of which monitor type they're attached to. This is what lets us add ML monitoring without touching any of the existing alert infrastructure.

Prompt 6:

"Would a Flask server work for the sidecar or is there a better option for exposing metrics like this?"

Response:

Flask is a solid choice for this. You need a lightweight HTTP server that exposes a couple of REST endpoints - /health for container health checks and /metrics for the Go monitor to poll. Flask handles that cleanly without pulling in a heavy framework. FastAPI would also work and gives you automatic OpenAPI docs, but for two endpoints that's overkill. Django is way too heavy for a sidecar. Flask with a single app.py file is the right level of complexity.

The /metrics handler would run the Evidently test suite against your sliding window of collected samples, compute the aggregate drift score, and return the JSON payload with all the fields matching the MLModelUsage struct - drift_score, confidence_avg, prediction_rate, model_version, and last_updated. The field names in the JSON need to match exactly because the Go monitor unmarshals directly into the struct. The sliding window approach works well here - you maintain the last 100 samples in memory, and on each /metrics call you run the Evidently comparison against the reference baseline. Stateless across container restarts, which is fine for a monitoring sidecar.

Prompt 7:

"i'll have a working vllm up and running, is it easy to get real data from it?"

Response:

Yeah, once vLLM is running it exposes Prometheus metrics at /metrics on the same port. The scraping function already follows the right pattern so it would be a matter of pointing it at the real endpoint instead of generating synthetic data. The metric names might differ slightly from what we've hardcoded - you'd want to check vLLM's actual output format - but the parsing logic for Prometheus text format is the same regardless. Shouldn't require major changes.

Prompt 8:

"How are the existing monitors connected to the ResourceManager? Where would I need to add the new MLModel one?"

Response:

Looking at resource.go, there are four methods where each monitor type integrates with the ResourceManager:

1. Run() - this is where the monitor goroutine gets started. CPU, Disk, and Memory each have their Start() called here. You need to add the same for MLModel.
2. Update() - this accepts configuration changes from fleet spec updates. There's a type switch on the monitor spec type. You need a new case for MLModelResourceMonitorSpec that creates or updates the MLModel monitor with the new sidecar endpoint, sampling interval, and alert rules.
3. Alerts() - this aggregates alerts from all active monitors into a single slice. You need to append MLModel alerts to the list.

4. Status() - this reports the current resource values back to the API server for display. You need to include the ML model drift score here, but only when the monitor is configured and has data.

The pattern is consistent across all four methods so once you've done one you'll see how the others follow. Start with Update() since that's where the monitor actually gets created, then wire up Run(), Alerts(), and Status() in that order. You'll also need to update the OpenAPI schema and run make generate to add the MLModel status field to the device status response - that's about 6 files total when you include the generated code.

Prompt 9:
"Make the MLModel field optional. If a device doesn't have ML monitoring configured it shouldn't show empty values."

Response:
Good call. I've changed the MLModel field from a direct MLModelMonitor value to a pointer (*MLModelMonitor) in the ResourceManager struct. Now all four integration methods check for nil before operating on it - Run() only starts the goroutine if the monitor exists, Update() creates the monitor on first configuration, Alerts() skips the ML model if it's nil, and Status() only includes ML model data in the response when the monitor is actively configured and has collected data.

This makes sense architecturally. CPU, memory, and disk are universal - every device has these resources and they're always monitored. But ML workloads are specialised. Most edge devices in a fleet probably aren't running inference servers, so having a mandatory ML model section in their status output with zero values would be confusing and wasteful. With the pointer approach, the field simply doesn't appear in the API response unless the fleet spec explicitly configures ML monitoring for that device.

Prompt 10:

"How does the status handler work differently for MLModel compared to CPU and Memory? The MLModel resource is optional but the others are always present."

Response:

The difference comes down to the Go type. CPU, Memory, and Disk use direct DeviceResourceStatusType values - they're always populated because every device has these resources. The status handler just sets the value directly.

MLModel uses a pointer (*DeviceResourceStatusType) so the API can distinguish between three states: "this device doesn't have ML monitoring configured" (nil pointer, field absent from JSON), "monitoring is configured but the sidecar hasn't reported yet" (pointer allocated but no data), and "monitoring is active and the current status is Healthy/Warning/Critical" (pointer to the actual status value).

In the setStatusFn implementation, it needs to check if the pointer is nil first. If it is, it allocates a new DeviceResourceStatusType and sets the pointer before assigning the value. Without that nil check, you'd get a nil pointer dereference panic the first time the monitor tries to report status. The CPU and memory handlers don't need this check because their fields are always pre-allocated.

Prompt 11:

"Can I replicate the existing tests as we did with the other code, if so do you have suggestions of what to test for"

Response:

Yeah, same approach works. The existing monitor tests in the repo follow a consistent pattern - httptest.Server for mocking endpoints, table-driven subtests for edge cases, and explicit error scenario coverage. You can replicate that structure and tailor the test cases to what's specific about the ML model monitor.

I'd suggest three categories:

First, a full lifecycle test - spin up an httptest.Server that returns a configurable drift score, create a monitor with warning at 30% and critical at 70%, start the collection loop, and validate behaviour at each stage. Set the drift score to 0.75 (75%), wait for a collection cycle, verify that both warning and critical alerts fire. Then drop it to 0.1 (10%), wait for another cycle, confirm all alerts clear. That covers the complete flow from HTTP collection through drift conversion to alert evaluation and clearing.

Second, edge case testing for the percentage conversion function specifically. This is a good candidate for a table-driven test. You'd want cases for zero (0.0 maps to 0), one (1.0 maps to 100), mid-range (0.5 maps to 50), rounding boundaries (0.764 should round down to 76, 0.765 should round up to 77), negative input (-0.1 should clamp to 0), and overflow (1.5 should clamp to 100). The rounding boundaries are worth testing explicitly because math.Round uses banker's rounding in some edge cases and you want to verify the behaviour is what you expect.

Third, HTTP failure scenarios for the collector. Things like: server returning HTTP 500 (should return an error, not panic), invalid JSON body (malformed response from a misconfigured sidecar), valid JSON but missing the required drift_score field, connection refused (sidecar

container not running), request timeout (sidecar hanging), and empty response body. The key assertion for all of these is that the collector returns a descriptive error rather than crashing, so the monitor can retry on the next interval.

Prompt 12:

"is there anything else reuiqred here, are there any internal flightctl tests/validation etc that we can need to check while i have the instance up, check the docs"

Response:

I checked the FlightCtl docs and the main validation command is make test, which runs the full unit test suite. You should run that on the EC2 instance while it's up to make sure nothing regressed. Beyond unit tests, proper integration testing would require a running FlightCtl instance with an enrolled device where you can actually apply a fleet spec with ML monitoring configured, deploy the drift sidecar, and verify the metrics flow end-to-end from sidecar through the agent to the API. That's a more involved setup and would come later once we have the sidecar container and deployment pipeline working.

Prompt 13:

"I want to use Evidently AI for drift detection in the sidecar. Show me the basic pattern for using TestSuite with TestColumnDrift to compare current inference metrics against a baseline."

Response:

The basic pattern in Evidently looks like this. You create a TestSuite and pass it a list of tests - in your case TestColumnDrift for each metric column you want to monitor (e.g., latency, throughput, confidence). Then you call suite.run() with two arguments: reference_data (a

pandas DataFrame representing your baseline - what "healthy" looks like) and current_data (another DataFrame with the latest samples you've collected).

After it runs, you call suite.as_dict() to get the results as a Python dictionary. This contains the test outcomes including whether each column passed or failed the drift test, and the statistical scores used to make that determination. The reference data should have enough samples for statistical significance - Evidently recommends at least 50-100 rows. You'd typically load this from a JSON file at startup representing the metrics you observed when the model was performing normally.

Prompt 14:

"how do I extract the drift score from the results? The as_dict() method returns a weird structure."

Response:

There are two approaches. You can access the nested structure directly if you know the exact layout - something like results['tests'][0]['parameters']['score'] - but this is fragile because if Evidently changes their output format in a future version, your code breaks silently.

The safer approach is to iterate through the test results using .get() with sensible defaults at each level. Start with results.get('tests', []), then for each test check test.get('status'), test.get('parameters', {}).get('score'). This way if a key is missing you get None instead of a KeyError, and you can handle it gracefully. I'd recommend this approach since you don't control Evidently's release cycle and you don't want a library update to crash your sidecar in production.

Prompt 15:

"what data does evidently actually need to perform proper drift calculations? are any of these specific indicators of drift or am i being too tight in my meaning for model health. open to expand it to any useful metrics which is what you seem to be doing?"

Response:

Good question, and there's an important distinction to unpack here.

True data drift in the ML sense means the distribution of your model's inputs or outputs has shifted compared to what it was trained on. To detect that properly, your sidecar would need to intercept the actual data flowing through the model - every user prompt going in, every response coming out - and compare those distributions against the training data distributions over time. That's architecturally much more invasive than what we're doing. You'd essentially need the sidecar sitting as a proxy between users and the inference server, capturing and analysing all traffic.

What you're getting from llama.cpp - tokens per second, latency, throughput - these are operational health metrics. They tell you how the model is performing computationally, not whether its outputs are drifting from what it learned during training. A latency spike doesn't mean the model is giving worse answers, it might mean the CPU is under load from something else entirely.

That said, Evidently doesn't care about the semantic meaning of the columns. It just does statistical comparison between two datasets. So you can absolutely feed it operational metrics and it will tell you if they've shifted significantly from the baseline. If your token generation rate drops by 40% compared to what you recorded as normal, Evidently will flag that as drift.

So it depends on how broadly you want to define "drift" for this project. You could frame it as: the sidecar monitors whatever metrics the end user determines are important for their model's health, and Evidently detects when those metrics deviate significantly from the established baseline. That's actually a more flexible and useful framing than strict data drift, because different deployments will care about different things.

Prompt 16:

"No, if I understand you correctly to detect true model drift our sidecar would need to sit in the middle of the user and inference comparing input and output to generate the score, this is probably far beyond the scale of this project?"

Response:

Correct - that would be a much larger undertaking. You'd be building an inference proxy that captures all request/response pairs, stores them, runs distribution analysis against training data, and manages the storage overhead of keeping all that data. That's a significant piece of infrastructure in its own right and well beyond the scope of a college project.

The sidecar pattern you have is the right approach for operational metrics. It's non-invasive, it doesn't sit in the critical path of inference traffic, and it's the same pattern that production monitoring systems like Datadog and New Relic use - scrape metrics from an endpoint periodically rather than intercepting traffic. The flexibility of your design, where the sidecar can be pointed at any metrics endpoint and configured to scrape whatever the user cares about, is arguably more useful than hardcoding it to only detect true data drift. Different edge deployments will have different concerns and this lets the operator decide what matters.

Prompt 17:

"How should I structure a Flask app that exposes metrics on port 5001? The Flightctl agent will poll /metrics every 30 seconds. I need to return JSON with drift_score, confidence_avg, prediction_rate, model_version, and last_updated."

Response:

I'd set it up with two endpoints. /health is a simple GET that returns {"status": "ok"} with a 200 - this is for container health checks so Kubernetes or systemd can verify the sidecar is alive. /metrics is the main endpoint that the Go monitor polls every samplingInterval seconds.

The /metrics handler should collect the latest sample from the inference server, add it to your sliding window, run the Evidently test suite against the window and the reference baseline, compute the aggregate drift score, and return the full JSON payload. The field names in the response need to match exactly what the MLModelUsage struct in mlmodel.go expects: drift_score as a float, confidence_avg as a float, prediction_rate as an integer, model_version as a string, and last_updated as a timestamp string. If any of the optional fields aren't available, just omit them from the JSON and the Go side will leave the corresponding pointer as nil.

Prompt 18:

"no we're dropping vllm fully, it's super complex and has a huge barrier to entry, is terrible on cpu. Bootc image can be either rhel 9 or centos or fedora, check the flgihtctl docs and i'll work with whatever version we find. I have a hugging face token to get the model we need."

Response:

Makes sense. vLLM is designed for GPU-accelerated serving with CUDA and it really struggles on CPU-only hardware - the performance would be terrible for a demo and the setup complexity

with GPU drivers on edge devices is significant. llama.cpp is purpose-built for CPU inference, supports quantised models to reduce memory requirements, and compiles from source without any special hardware dependencies. Much more practical for edge deployment.

I checked the FlightCtl docs and CentOS Stream 9 is the recommended base for bootc images. For the application stack we'll need three containers: a modelcar that holds the model weights as an OCI artifact, an inference server running llama.cpp compiled from source, and an OpenWebUI frontend for the chat interface. We can build each one separately and deploy them through the fleet specification.

Prompt 19:

"I need to switch the modelcar from safetensors to GGUF format for llama.cpp. The quantised format Q4_K_M is what I need."

Response:

Updated the model downloader script to pull the GGUF quantised version instead of safetensors. llama.cpp uses GGUF as its native model format - it can't load safetensors directly, those are for PyTorch/Transformers. Q4_K_M is a good quantisation level for your use case - it gives roughly 4-bit precision which cuts the model size significantly while maintaining decent output quality. The download pattern is slightly different since quantised GGUF files are often published as separate repos or in specific branches on HuggingFace rather than being part of the main model repo.

Prompt 20:

"claud how do you think it has 260gb of build cache its an ec2 instance you’re meant to be debugginh"

Response:

You're right, sorry about that - I was looking at the local Docker info on your machine rather than the EC2 instance. A fresh EC2 instance wouldn't have 260GB of build cache, that's obviously your local development machine's accumulated cache. I need to SSH in and check the actual disk usage there rather than referencing local state.

Prompt 21:

"are you getting confused between my local machine and the ec2?? double check everything"

Response:

Apologies, yes I was mixing up the two environments. Let me be clear about the distinction going forward:

Your local machine is Apple Silicon Mac. make test works here for running unit tests, but make build fails because several container images in the FlightCtl build pipeline (like quay.io/openshift/origin-cli:4.20.0) don't have ARM64 variants, and some Containerfiles hardcode x86_64 RPM URLs.

The EC2 instance is x86_64 RHEL 9. This is where the full make build, make deploy, and all container builds need to happen. It's also where the FlightCtl instance runs in KIND.

I'll make sure all commands going forward are explicitly targeting the correct environment. The context compaction may have caused me to lose track of which environment we were working in.

Prompt 22:

[Pasted screnshot output from KIND failing with Podman on EC2]

Response:

The exit code 137 means the control-plane container is being killed by the OOM killer. This is a known issue with KIND v0.29.0 running on rootless Podman - there's a conflict in how Podman manages cgroups that causes the KIND node containers to exceed their memory allocation and get killed by the kernel.

I looked at FlightCtl's own CI pipeline to see what they use, and they pin KIND to v0.22.0 and run it with Docker, not Podman. That's a much more stable combination. I'd recommend switching to Docker for the KIND cluster and downgrading KIND to v0.22.0 to match what FlightCtl's CI tests against. This way you're running on a configuration that's known to work with this codebase.

Prompt 23:

"this is not true, you can define quadlets in flightctl, please examine the docs"

Response:

You're right, I apologise for the error. I've re-read the FlightCtl documentation and it does support Quadlet definitions directly in fleet specs. You can define individual Quadlet units inline in the fleet YAML under the applications section with appType set to the appropriate Quadlet type. I was wrong to say it wasn't supported - I should have checked the docs before making that claim.

Prompt 24:

"lets exmaine the metrics coming out of the device from llama cpp"

Response:

Looking at the llama.cpp /metrics endpoint on the running device, it exposes Prometheus-format text metrics. The main ones relevant for drift detection are prompt_tokens_seconds (the rate at which it processes input tokens - essentially how fast it reads your prompt), predicted_tokens_seconds (the rate at which it generates output tokens - how fast it produces the response), and several other counters for total requests, active slots, and cache usage. The token rate metrics are the most useful for operational drift detection because they directly reflect the model's computational performance. If predicted_tokens_seconds drops significantly from the baseline, it could indicate the model is struggling - maybe the hardware is degrading, maybe a new model version is slower, or maybe there's resource contention from other workloads.

Prompt 25:

"Rewrite the drift monitor to scrape llama.cpp instead of vLLM. llama.cpp exposes predicted_tokens_seconds and prompt_tokens_seconds on its /metrics endpoint."

Response:

Done. I've rewritten the scrape function completely. Instead of parsing vLLM's JSON API format, it now fetches the Prometheus text format from llama.cpp's /metrics endpoint and parses it line by line looking for the prompt_tokens_seconds and predicted_tokens_seconds gauges. Each scrape adds a new row to the sliding window DataFrame with the current values and a timestamp.

I've also updated reference_data.json with baseline values that actually match what llama.cpp produces. The ranges are quite different from vLLM - llama.cpp on CPU reports much lower token rates than vLLM on GPU, so the old baseline values would immediately trigger false drift alerts. The new baseline has 50 samples representing healthy CPU inference performance that I derived from typical llama.cpp benchmarks for the 1B parameter model on commodity hardware.

Prompt 26:

"I've no interest in waiting for 20 scrapes, is this a requirement from the evidently library or is it arbitrary"

Response:

It's not arbitrary - Evidently needs a minimum number of data points to perform statistically meaningful drift calculations. The statistical tests it uses (like the Kolmogorov-Smirnov test for numerical columns) require enough samples to estimate the distribution reliably. With fewer than about 20 samples, the test results would be unreliable - you'd get false positives where normal variation looks like drift, or false negatives where real drift gets missed because the sample is too small to detect it.

That said, this only affects the real drift calculation path. The /simulate endpoint bypasses Evidently entirely - it just sets a hardcoded drift value in a global variable that /metrics returns directly. So for your demo and video recording, you can trigger alerts immediately with a curl command without waiting for any scraping.

Prompt 27:

"Thats my next question. I'll need to show it fire at both other levels so whats our best approach, do we create two separate little scripts one that will trigger the warning to fire, and one that will trigger the critical alert to fire, I need to demonstrate this for the video i have to record for submission"

Response:

I'd suggest creating shell scripts for each scenario. You could have trigger-warning.sh that sets an environment variable to override the drift score to 0.35 (above the 30% warning threshold but below 70% critical), and trigger-critical.sh that sets it to 0.85. A third script reset-normal.sh would clear the override and return to real metrics. Each script would need to restart the sidecar container with the new environment variable...

Prompt 28:

"That's a helluva lot of shell scripts, sure there is a better approach, can I just curl those endpoints?"

Response:

Yes, much simpler. Add a /simulate endpoint to the Flask app that accepts a level parameter via query string. When you curl localhost:5001/simulate?level=critical it sets a global variable to 0.9 (above your 70% critical threshold). When you curl /simulate?level=warning it sets it to 0.4 (above warning, below critical). The /metrics endpoint checks this variable first - if it's set, return the override value instead of running the Evidently suite. Then /simulate/reset clears it and goes back to real calculations. No scripts, no container restarts, just curl commands you can run live during the demo.

Prompt 29:

"I need to connect the MLModel alerts through Flightctl's alert exporter. Show me how the existing CPU, Memory, and Disk events are structured in event_processor.go."

Response:

Looking at event_processor.go, the pattern for each resource type is consistent. Each has three event reason constants - for example EventReasonDeviceCPUCritical, EventReasonDeviceCPUWarning, EventReasonDeviceCPUNormal. These are grouped together in a slice variable (cpuGroup) that the alert lifecycle functions use. When a threshold is breached, setAlert creates an alert with the appropriate reason. When it returns to normal, clearAlertGroup clears all alerts in that group.

For MLModel you need the same three constants (EventReasonDeviceMLModelCritical, EventReasonDeviceMLModelWarning, EventReasonDeviceMLModelNormal), a group variable, and the corresponding case handlers in the processEvent switch statement. I've also checked device.go on the server side - that needs three new status message constants (MLModelIsCritical, MLModelIsWarning, MLModelIsNormal) and a resource check function. The implementation is the same pattern but with one key difference: MLModel uses pointer types for the status fields since it's optional, so every access needs a nil check before dereferencing.

Prompt 30:

"how will we verify the alerter fired? it isn't onnected to anything yet"

Response:

Right now the alert fires into Alertmanager but Alertmanager has no receiver configured for it - it'll just sit there as a firing alert that nobody sees. You need something on the receiving end to

prove the full pipeline works. A few options: you could add a Slack webhook (quick but requires a Slack workspace), set up a simple HTTP listener pod that logs incoming webhooks (works but you'd need to check the logs), or connect it to a GitHub Action via repository_dispatch that logs the payload (more work to set up but gives you a visible, shareable record of the alert firing). Any of those would close the loop and let you verify end-to-end.

Prompt 31:

"The MLModel monitor config is being rejected by the API. Debug this - the other monitors work fine."

Response:

Found it. In validation.go there's a switch statement that validates incoming resource monitor configurations. It has explicit cases for CPU, Memory, and Disk that perform the type-specific validation, but there's no case for MLModel. When you submit a fleet spec with an MLModel monitor, the switch falls through to the default case which returns a "unsupported monitor type" validation error. The fix is adding a case for the MLModel type that validates the sidecar endpoint URL and alert rules. Seven lines of code - just matching the pattern of the existing cases with the MLModel-specific fields.

Prompt 32:

"I want it to connect to a github action or similar in this repo that will serve as a fake webhook into rhoai"

Response:

Good idea - that gives you a visible, verifiable record of the alert triggering and it demonstrates the concept of connecting to a retraining pipeline. There's one complication though: Alertmanager's native webhook format doesn't match what GitHub Actions expects. Alertmanager sends a POST with a JSON body containing a status field and an alerts array where each alert has labels, annotations, startsAt, and endsAt. GitHub's repository_dispatch API is completely different - it expects a POST to /repos/{owner}/{repo}/dispatches with an event_type string and a client_payload object.

So you'll need two pieces:

1. A GitHub Actions workflow file in the repo that triggers on repository_dispatch events with your custom event type, logs the payload, and simulates the retraining step
2. A small bridge service - another Flask app - that sits between Alertmanager and GitHub. It receives the Alertmanager webhook, checks if it's an MLModel alert, extracts the relevant information, and makes the GitHub API call with the right format

The bridge is essentially a format translator between two systems that were never designed to talk to each other directly.

Prompt 33:

"yeah trigger a critical drift for me and lets check the it work"

Response:

Running curl http://localhost:5001/simulate?level=critical on the device. That sets the drift score to 0.9, which is 90% - well above your 70% critical threshold. Now let's watch the pipeline:

The agent polls /metrics every 30 seconds, so it should pick up the elevated score on the next cycle. Once it does, it evaluates the score against the alert rules, sees it exceeds the critical threshold with the configured duration, and writes a Critical event to the database. The alert exporter polls the event table every 60 seconds, converts the event to a Prometheus alert, and pushes it to Alertmanager. Alertmanager matches it against the webhook receiver configuration and fires the webhook to our bridge. The bridge filters for MLModel alerts, extracts the payload, and dispatches to GitHub via repository_dispatch. The GitHub Actions workflow triggers and logs the payload.

Let me check each step... the agent has picked it up, event is in the database, alert exporter has forwarded it, Alertmanager shows a firing alert, and checking GitHub - the Actions workflow has triggered. Full pipeline confirmed working.

Prompt 34:

"yes, the whole thing works, lets reset the monitor and make sure it goes back to normal"

Response:

Running curl http://localhost:5001/simulate/reset on the device. This clears the simulation override and the next /metrics call will return the drift score of 0.05 (the default when no real drift is detected), which is 5% - well below the 30% warning threshold. On the next agent poll cycle it should report the normal score, the event processor creates a Normal event that clears the alert group, Alertmanager resolves the firing alert, and the device status goes back to healthy.

Let me verify... confirmed. The agent is reporting the normal score, the alert has resolved in Alertmanager, and the device status in FlightCtl shows healthy across all resources including ML model. The full lifecycle works - alert fires on drift, clears on recovery.

Prompt 35:

"tested end to end and confirmed working"

Response:

That's the complete pipeline verified end-to-end: drift sidecar produces a score, agent collects it, alert rules evaluate it, event processor records the state change, alert exporter forwards to Alertmanager, Alertmanager fires the webhook, bridge translates and dispatches to GitHub, and the Actions workflow runs. And the reverse path works too - alert clears when the score normalises. Let me document all of this in the development log with the exact commands used and what we observed at each stage so it's reproducible.

Prompt 36:

"what format does the alertmanger webhook match? What it is desgined to integrate into?"

Response:

Alertmanager's webhook receiver sends a POST request with a JSON body that has a specific structure. At the top level there's a status field ("firing" or "resolved"), a groupLabels object, and an alerts array. Each alert in the array has its own status, a labels map (containing alertname, severity, device, etc.), an annotations map (containing summary and description text), and startsAt/endsAt timestamps.

This format is designed for notification platforms. Slack, PagerDuty, OpsGenie, email gateways, and similar alerting tools all have native Alertmanager integrations that understand this payload format. They parse the labels to determine routing, use the annotations for the notification body, and track the startsAt/endsAt for alert duration.

GitHub Actions is not one of those systems. It has no Alertmanager integration. GitHub expects repository_dispatch events, which are a completely different structure - a POST to the GitHub API with an event_type string and a client_payload JSON object. The format, the endpoint, the authentication mechanism - all different. That's exactly why the webhook bridge exists: it's translating between two systems that were never designed to communicate with each other.

Prompt 37:

"I need you to put a Created by Shaun Walsh on any original files created by me here, or if I modified existing files please indicate where the changes were made with my name at the start and end of each section. This will allow my examiners to simple search for my name within the file and examine accoridngly"

Response:

Understood. I'll go through every file we've created or modified on this branch. For files that are entirely new (mlmodel.go, mlmodel_test.go, drift_monitor.py, webhook_bridge.py, all the Containerfiles, the fleet specs, the GitHub Actions workflow), I'll add a comment at the top of each file: "Created by Shaun Walsh" in the appropriate comment syntax for that language.

For existing files where we added code (resource.go, event_processor.go, device.go, validation.go, openapi.yaml), I'll add "Modified by Shaun Walsh - changes begin" before our additions and "Modified by Shaun Walsh - changes end" after them. This way an examiner can search for "Shaun Walsh" across the entire repo and immediately find every piece of code you contributed.

I'll skip the auto-generated files (spec.gen.go, types.gen.go) since those are machine-produced by make generate and re-annotating them would be misleading.